```
using System;
using System.Collections.Generic;
using System.Linq;
using FW.Procedure;

namespace FW
{
	internal sealed class FSM<T> : FSMBase, IReference, IFSM<T> where T : class
	{
		private T _Owner;
		private readonly Dictionary<Type, FSMState<T>> _States;
		private FSMState<T> _CurrentState;

		public FSM()
		{
			_Owner = null;
			_States = new Dictionary<Type, FSMState<T>>();
			_CurrentState = null;
		}

		public static FSM<T> Create(string name, T owner, params FSMState<T>[] states)
		{
			if (owner == null)
			{
				throw new Exception("FSM owner is invalid.");
			}

			if (states == null || states.Length == 0)
			{
				throw new Exception("FSM states is invalid.");
			}

			FSM<T> fsm = ReferencePool.Acquire<FSM<T>>();
			fsm.Name = name;
			fsm._Owner = owner;

			foreach (var state in states)
			{
				if (state == null)
				{
					throw new Exception("FSM add state is invalid.");
				}

				var stateType = state.GetType();
				if (fsm._States.ContainsKey(stateType))
				{
					throw new Exception(Utils.Text.Format("FSM '{0}' state '{1}' is
already exist.", name, stateType));
				}

				fsm._States.Add(stateType, state);
				state.OnInit(fsm);
			}
```

```
        return fsm;
    }

    void IReference.Clear()
    {
        if (_CurrentState != null)
        {
            _CurrentState.OnLeave(this, true);
        }

        foreach (var state in _States.Values)
        {
            state.OnDestroy(this);
        }
        _States.Clear();

        _Owner = null;
        _CurrentState = null;
    }

    internal override void Update(float deltaTime, float unscaledDeltaTime)
    {
        if (_CurrentState == null)
        {
            return;
        }

        _CurrentState.OnUpdate(this, deltaTime, unscaledDeltaTime);
    }

    internal override void Destroy()
    {
        ReferencePool.Release(this);
    }

    T IFSM<T>.Owner => _Owner;

    FSMState<T> IFSM<T>.CurrentState => _CurrentState;

    void IFSM<T>.Start<TState>()
    {
        var state = GetState<TState>();
        if (state == null)
        {
            return;
        }

        _CurrentState = state;
        object data = (_Owner as IProcedureManager)?.GetAndClearProcedureData();
        _CurrentState.OnEnter(this, data);
    }
```

```
            void IFSM<T>.ChangeState<TState>()
            {
                  if (_CurrentState == null)
                  {
                        throw new Exception("Current state is invalid.");
                  }

                  var state = GetState<TState>();
                  if (state == null)
                  {
                        return;
                  }

                  _CurrentState.OnLeave(this, false);
                  _CurrentState = state;
                  object data = (_Owner as IProcedureManager)?.GetAndClearProcedureData();
                  _CurrentState.OnEnter(this, data);
            }

            private TState GetState<TState>() where TState : FSMState<T>
            {
                  var stateType = typeof(TState);
                  if (_States.TryGetValue(stateType, out var state))
                  {
                        return state as TState;
                  }

                  throw new Exception(Utils.Text.Format("FSM '{0}' can not start state '{1}'
which is not exist.",
                        typeof(T), typeof(TState)));
            }
      }
}

using System;
using System.Collections.Generic;

namespace FW
{
      internal class FSMManager : ModuleBase, IFSMManager
      {
            private readonly Dictionary<string, FSMBase> _FSMs = new();

            private List<string> _destroyedFSMAfterUpdate = new();

            internal override int Priority => 60;

            internal override void Update(float deltaTime, float unscaledDeltaTime)
            {
                  if (_FSMs.Count == 0)
                  {
                        return;
                  }
```

```
                foreach (var fsm in _FSMs.Values)
                {
                        fsm.Update(deltaTime, unscaledDeltaTime);
                }

                // 延迟销毁 FSM，避免在更新过程中销毁导致的问题
                foreach (var fsmName in _destroyedFSMAfterUpdate)
                {
                        if (_FSMs.TryGetValue(fsmName, out var fsm))
                        {
                                fsm.Destroy();
                                _FSMs.Remove(fsmName);
                        }
                }
                _destroyedFSMAfterUpdate.Clear();
        }

        internal override void Destroy()
        {
                foreach (var fsm in _FSMs.Values)
                {
                        fsm.Destroy();
                }

                _FSMs.Clear();
        }

        IFSM<T> IFSMManager.CreateFSM<T>(string name, T owner, params FSMState<T>[] states)
        {
                if (HasFSM(name))
                {
                        throw new Exception(
                                Utils.Text.Format("FSMManager create fsm failed, fsm '{0}' is
already exist.", name));
                }

                var fsm = FSM<T>.Create(name, owner, states);
                _FSMs.Add(name, fsm);

                return fsm;
        }

        bool IFSMManager.DestroyFSM(string name)
        {
                _destroyedFSMAfterUpdate.Add(name);
                return true;
        }

        private bool HasFSM(string name)
        {
                return _FSMs.ContainsKey(name);
        }
```

```
	}
}
、
using System;
using System.Collections.Generic;
using System.Reflection;
using Cysharp.Threading.Tasks;
using UnityEngine;

namespace FW
{
	public class UIManager : MonoBehaviour, IUIManager
	{
		[Header("World Space Canvas")]
		[SerializeField] private Canvas worldUICanvas;

		[Header("UI 相机")]
		[SerializeField] private new Camera camera;

		[Header("切换场景的时候可能没有相机的时候临时替换用")]
		[SerializeField] private Camera changeSceneCamera;

		[Header("层级数组")]
		[SerializeField] private GameObject[] layers;

		private IResManager _resManager;

		private readonly Dictionary<string, UIBase> _UIs = new();
		private readonly Dictionary<string, UnityEngine.Object> _Refs = new();

		private readonly HashSet<string> _loadings = new();

		#region unity

		private void Update()
		{
			ProcCamera();

			UpdateUI();
		}

		private void OnDestroy()
		{
			_UIs.Clear();
		}

		#endregion

		#region assistent

		private void ProcCamera()
		{
			var mainCamera = Camera.main;
```

```
                if (mainCamera == null)
                {
                        changeSceneCamera.enabled = true;
                }
                else
                {
                        changeSceneCamera.enabled = false;
                        worldUICanvas.worldCamera = mainCamera;
                        mainCamera.AddCameraToStack(camera);
                }
        }

        private TUI Get<TUI>(string uiAddress) where TUI : UIBase
        {
                _UIs.TryGetValue(uiAddress, out var uiBase);
                return uiBase as TUI;
        }

        #endregion

        #region interface

        public void Init(IResManager resManager)
        {
                _resManager = resManager;
        }

        public async UniTask<TUI> Open<TUI>(string uiAddress, UIModelBase model) where TUI :
UIBase, new()
        {
                var ui = Get<TUI>(uiAddress);
                if (ui != null)
                {
                        Log.Error($"UIManager want to open exist ui {uiAddress}");
                        return ui;
                }

                if (_loadings.Contains(uiAddress))
                {
                        return null;
                }

                _loadings.Add(uiAddress);

                var go = await _resManager.LoadAssetAsync<GameObject>(uiAddress);

                // 已经关闭了（界面有可能在还没有加载完成的时候关闭）
                if (!_loadings.Contains(uiAddress))
                {
                        _resManager.Release(go);
                        return null;
                }
```

```
                var config = go.GetComponent<UIConfig>();
                var layer = layers[config.Layer].transform;

                var inst = Instantiate(go, layer);
                if (inst == null)
                {
                        Log.Error($"UIManager load an object that is not ui {uiAddress}");
                        return null;
                }

                ui = new TUI();
                if (ui == null)
                {
                        throw new TypeAccessException(Utils.Text.Format("UIManager create ui
type incorrect {0} {1}", uiAddress));
                }

                var createMethod = ui.GetType().GetMethod("Create", BindingFlags.Instance |
BindingFlags.NonPublic);
                createMethod?.Invoke(ui, new object[] {inst, model});

                var openMethod = ui.GetType().GetMethod("Open", BindingFlags.Instance |
BindingFlags.NonPublic);
                openMethod?.Invoke(ui, null);

                // ui.Open(inst, model);

                _UIs.Add(uiAddress, ui);
                _Refs.Add(uiAddress, go);

                return ui;
        }

        public void UpdateUI()
        {
                foreach (var ui in _UIs.Values)
                {
                        var updateMethod = ui.GetType().GetMethod("Update",
BindingFlags.Instance | BindingFlags.NonPublic);
                        updateMethod?.Invoke(ui, null);

                        // ui.Update(deltaTime);
                }
        }

        public void Close(string uiAddress)
        {
                _loadings.Remove(uiAddress);

                var uiBase = Get<UIBase>(uiAddress);
                if (uiBase == null)
                {
                        Log.Error($"UIManager close not exist ui {uiAddress}");
```

```
                return;
            }

            if (_Refs.Remove(uiAddress, out var obj))
            {
                _resManager.Release(obj);
            }

            _UIs.Remove(uiAddress);

            var closeMethod = uiBase.GetType().GetMethod("Close", BindingFlags.Instance
| BindingFlags.NonPublic);
            closeMethod?.Invoke(uiBase, null);

            // uiBase.Close();
        }

        public void Hide(string uiAddress)
        {
            var uiBase = Get<UIBase>(uiAddress);
            if (uiBase == null)
            {
                Log.Error($"UIManager hide not exist ui {uiAddress}");
                return;
            }

            var hideMethod = uiBase.GetType().GetMethod("Hide", BindingFlags.Instance |
BindingFlags.NonPublic);
            hideMethod?.Invoke(uiBase, null);

            // uiBase.Hide();
        }

        public void Show(string uiAddress)
        {
            var uiBase = Get<UIBase>(uiAddress);
            if (uiBase == null)
            {
                Log.Error($"UIManager show not exist ui {uiAddress}");
                return;
            }

            var showMethod = uiBase.GetType().GetMethod("Show", BindingFlags.Instance |
BindingFlags.NonPublic);
            showMethod?.Invoke(uiBase, null);

            // uiBase.Show();
        }

        #endregion

    }
}
```

```
using UnityEngine;

namespace XZ.Input
{
	/// <summary>
	/// 手柄输入提供者
	/// 支持 Xbox、PlayStation、Switch 等多种手柄
	/// </summary>
	public class GamepadInputProvider : IInputProvider
	{
		public InputMode InputMode => InputMode.Gamepad;
		public bool IsActive { get; private set; }
		public float DeadZone { get; set; }

		private int joystickIndex = 0; // 手柄索引（支持多手柄）

		public GamepadInputProvider(float deadZone = 0.2f, int joystickIndex = 0)
		{
			DeadZone = deadZone;
			this.joystickIndex = joystickIndex;
		}

		public void Update()
		{
			// 检测手柄是否连接
			string[] joysticks = UnityEngine.Input.GetJoystickNames();
			IsActive = joysticks.Length > joystickIndex
&& !string.IsNullOrEmpty(joysticks[joystickIndex]);
		}

		public Vector2 GetAxis(string axisName)
		{
			switch (axisName)
			{
				case "Move":
					return GetMoveAxis();

				case "Look":
					return GetLookAxis();

				default:
					return Vector2.zero;
			}
		}

		private Vector2 GetMoveAxis()
		{
			// 左摇杆
			float horizontal = UnityEngine.Input.GetAxis("Horizontal");
			float vertical = UnityEngine.Input.GetAxis("Vertical");
```

```
            Vector2 axis = new Vector2(horizontal, vertical);

            // 应用死区
            if (axis.magnitude < DeadZone)
            {
                  return Vector2.zero;
            }

            // 保持圆形死区
            if (axis.magnitude > 1f)
            {
                  axis.Normalize();
            }

            return axis;
      }

      private Vector2 GetLookAxis()
      {
            // 右摇杆 (Unity 标准映射)
            // 尝试多种可能的轴名称，因为不同手柄映射可能不同
            float horizontal = 0f;
            float vertical = 0f;

            // 优先尝试标准映射
            horizontal = UnityEngine.Input.GetAxis("RightStickHorizontal");
            vertical = UnityEngine.Input.GetAxis("RightStickVertical");

            // 如果标准映射不可用，尝试备用映射
            if (Mathf.Approximately(horizontal, 0f) && Mathf.Approximately(vertical,
0f))
            {
                  // 尝试使用轴 4 和 5（某些手柄的右摇杆）
                  horizontal = UnityEngine.Input.GetAxis("Axis 4");
                  vertical = UnityEngine.Input.GetAxis("Axis 5");
            }

            // 如果还是不可用，尝试轴 6 和 7
            if (Mathf.Approximately(horizontal, 0f) && Mathf.Approximately(vertical,
0f))
            {
                  horizontal = UnityEngine.Input.GetAxis("Axis 6");
                  vertical = UnityEngine.Input.GetAxis("Axis 7");
            }

            // 注意：不再回退到鼠标输入，因为这是手柄模式
            Vector2 axis = new Vector2(horizontal, vertical);

            // 应用死区
            if (axis.magnitude < DeadZone)
            {
                  return Vector2.zero;
            }
```

```
            return axis;
        }

        public bool GetButton(string buttonName)
        {
            switch (buttonName)
            {
                case "Jump":
                    return UnityEngine.Input.GetButton("Jump"); // A/X

                case "Attack":
                    return UnityEngine.Input.GetButton("Fire1"); // B/Circle 或 RT

                case "Dash":
                    return UnityEngine.Input.GetButton("Fire2"); // RB/R1

                case "Interact":
                    return UnityEngine.Input.GetButton("Fire3"); // Y/Triangle

                case "Pause":
                    // 使用 GetButton 保持一致性，但 Start 按钮通常不在标准 Input
Manager 中
                    // 所以使用 KeyCode 作为备用
                    return UnityEngine.Input.GetButton("Cancel") ||
                           UnityEngine.Input.GetKey(KeyCode.JoystickButton7);
// Start

                default:
                    return false;
            }
        }

        public bool GetButtonDown(string buttonName)
        {
            switch (buttonName)
            {
                case "Jump":
                    return UnityEngine.Input.GetButtonDown("Jump");

                case "Attack":
                    return UnityEngine.Input.GetButtonDown("Fire1");

                case "Dash":
                    return UnityEngine.Input.GetButtonDown("Fire2");

                case "Interact":
                    return UnityEngine.Input.GetButtonDown("Fire3");

                case "Pause":
                    // 使用 GetButtonDown 保持一致性
                    return UnityEngine.Input.GetButtonDown("Cancel") ||
```

```
                                    UnityEngine.Input.GetKeyDown(KeyCode.JoystickButton
7);

                default:
                        return false;
                }
        }

        public bool GetButtonUp(string buttonName)
        {
                switch (buttonName)
                {
                        case "Jump":
                                return UnityEngine.Input.GetButtonUp("Jump");

                        case "Attack":
                                return UnityEngine.Input.GetButtonUp("Fire1");

                        case "Dash":
                                return UnityEngine.Input.GetButtonUp("Fire2");

                        case "Interact":
                                return UnityEngine.Input.GetButtonUp("Fire3");

                        case "Pause":
                                // 使用 GetButtonUp 保持一致性
                                return UnityEngine.Input.GetButtonUp("Cancel") ||
                                        UnityEngine.Input.GetKeyUp(KeyCode.JoystickButton7)
;

                        default:
                                return false;
                }
        }

        /// <summary>
        /// 获取手柄类型
        /// </summary>
        public string GetGamepadName()
        {
                string[] joysticks = UnityEngine.Input.GetJoystickNames();
                if (joysticks.Length > joystickIndex)
                {
                        return joysticks[joystickIndex];
                }
                return "Unknown";
        }

        /// <summary>
        /// 获取扳机轴
        /// </summary>
        public float GetTriggerAxis(bool leftTrigger)
        {
```

```
                float value = 0f;

                // 尝试多种可能的轴映射
                if (leftTrigger)
                {
                    // 左扳机：尝试 Axis 3, Axis 9, 或标准映射
                    value = UnityEngine.Input.GetAxis("Axis 3");
                    if (Mathf.Approximately(value, 0f))
                    {
                        value = UnityEngine.Input.GetAxis("Axis 9");
                    }
                    if (Mathf.Approximately(value, 0f))
                    {
                        // 某些手柄使用标准轴名称
                        value = UnityEngine.Input.GetAxis("LeftTrigger");
                    }
                }
                else
                {
                    // 右扳机：尝试 Axis 6, Axis 10, 或标准映射
                    value = UnityEngine.Input.GetAxis("Axis 6");
                    if (Mathf.Approximately(value, 0f))
                    {
                        value = UnityEngine.Input.GetAxis("Axis 10");
                    }
                    if (Mathf.Approximately(value, 0f))
                    {
                        // 某些手柄使用标准轴名称
                        value = UnityEngine.Input.GetAxis("RightTrigger");
                    }
                }

                // 应用死区
                if (Mathf.Abs(value) < DeadZone)
                {
                    return 0f;
                }

                return value;
            }

            /// <summary>
            /// 振动手柄（如果支持）
            /// </summary>
            public void Vibrate(float intensity, float duration)
            {
                // Unity 原生不支持手柄振动
                // 需要使用平台特定的 API 或第三方插件
                Debug.Log($"手柄振动: 强度={intensity}, 时长={duration}");
            }
        }
    }
    using UnityEngine;
```

```
using System.Collections.Generic;

namespace XZ.Input
{
	/// <summary>
	/// 输入缓冲系统
	/// 用于改善操作手感，特别是在平台游戏中
	/// </summary>
	public class InputBuffer
	{
		private Dictionary<string, BufferedInput> bufferedInputs = new Dictionary<string,
BufferedInput>();
		private float bufferTime;

		public InputBuffer(float bufferTime = 0.2f)
		{
			this.bufferTime = bufferTime;
		}

		/// <summary>
		/// 设置缓冲时间
		/// </summary>
		public void SetBufferTime(float time)
		{
			bufferTime = time;
		}

		/// <summary>
		/// 记录输入
		/// </summary>
		public void RecordInput(string inputName, bool value)
		{
			if (value)
			{
				if (!bufferedInputs.ContainsKey(inputName))
				{
					bufferedInputs[inputName] = new BufferedInput
					{
						inputName = inputName,
						recordTime = Time.time,
						consumed = false
					};
				}
				else
				{
					// 更新记录时间
					bufferedInputs[inputName].recordTime = Time.time;
					bufferedInputs[inputName].consumed = false;
				}
			}
		}

		/// <summary>
```

```
        /// 检查是否有缓冲的输入
        /// </summary>
        public bool HasBufferedInput(string inputName)
        {
            if (!bufferedInputs.ContainsKey(inputName))
                return false;

            var buffered = bufferedInputs[inputName];

            // 检查是否过期
            if (Time.time - buffered.recordTime > bufferTime)
            {
                bufferedInputs.Remove(inputName);
                return false;
            }

            return !buffered.consumed;
        }

        /// <summary>
        /// 消费缓冲的输入
        /// </summary>
        public bool ConsumeBufferedInput(string inputName)
        {
            if (!bufferedInputs.ContainsKey(inputName))
                return false;

            var buffered = bufferedInputs[inputName];

            // 检查是否过期
            if (Time.time - buffered.recordTime > bufferTime)
            {
                bufferedInputs.Remove(inputName);
                return false;
            }

            // 如果已消费，返回 false
            if (buffered.consumed)
                return false;

            // 标记为已消费
            buffered.consumed = true;
            return true;
        }

        /// <summary>
        /// 清除所有缓冲的输入
        /// </summary>
        public void Clear()
        {
            bufferedInputs.Clear();
        }
```

```
            /// <summary>
            /// 清除过期的输入
            /// </summary>
            public void ClearExpired()
            {
                  var expiredKeys = new List<string>();

                  foreach (var kvp in bufferedInputs)
                  {
                        if (Time.time - kvp.Value.recordTime > bufferTime)
                        {
                              expiredKeys.Add(kvp.Key);
                        }
                  }

                  foreach (var key in expiredKeys)
                  {
                        bufferedInputs.Remove(key);
                  }
            }

            private class BufferedInput
            {
                  public string inputName;
                  public float recordTime;
                  public bool consumed;
            }
      }
}
using UnityEngine;
using System;
using System.Collections.Generic;
using System.IO;

namespace XZ.Input
{
      /// <summary>
      /// 输入录制和重放系统
      /// 用于测试和调试
      /// </summary>
      public class InputRecorder
      {
            private List<InputFrame> recordedFrames = new List<InputFrame>();
            private bool isRecording = false;
            private bool isReplaying = false;
            private int replayIndex = 0;
            private float recordStartTime;
            private float replayStartTime;

            /// <summary>
            /// 是否正在录制
            /// </summary>
            public bool IsRecording => isRecording;
```

```
        /// <summary>
        /// 是否正在重放
        /// </summary>
        public bool IsReplaying => isReplaying;

        /// <summary>
        /// 录制的帧数
        /// </summary>
        public int RecordedFrameCount => recordedFrames.Count;

        /// <summary>
        /// 开始录制
        /// </summary>
        public void StartRecording()
        {
            if (isRecording) return;

            isRecording = true;
            recordedFrames.Clear();
            recordStartTime = Time.time;
        }

        /// <summary>
        /// 停止录制
        /// </summary>
        public void StopRecording()
        {
            isRecording = false;
        }

        /// <summary>
        /// 录制一帧输入
        /// </summary>
        public void RecordFrame(Vector2 moveAxis, Vector2 lookAxis, Dictionary<string,
bool> buttonStates)
        {
            if (!isRecording) return;

            float time = Time.time - recordStartTime;

            var frame = new InputFrame
            {
                time = time,
                moveAxis = moveAxis,
                lookAxis = lookAxis,
                buttonStates = new Dictionary<string, bool>(buttonStates)
            };

            recordedFrames.Add(frame);
        }

        /// <summary>
```

```
        /// 开始重放
        /// </summary>
        public void StartReplay()
        {
            if (isReplaying || recordedFrames.Count == 0) return;

            isReplaying = true;
            replayIndex = 0;
            replayStartTime = Time.time;
        }

        /// <summary>
        /// 停止重放
        /// </summary>
        public void StopReplay()
        {
            isReplaying = false;
            replayIndex = 0;
        }

        /// <summary>
        /// 获取当前重放的输入
        /// </summary>
        public InputFrame? GetReplayFrame()
        {
            if (!isReplaying || recordedFrames.Count == 0)
                return null;

            float currentTime = Time.time - replayStartTime;

            // 查找当前时间对应的帧
            while (replayIndex < recordedFrames.Count)
            {
                var frame = recordedFrames[replayIndex];

                if (frame.time <= currentTime)
                {
                    replayIndex++;
                    return frame;
                }
                else
                {
                    break;
                }
            }

            // 如果已经播放完所有帧
            if (replayIndex >= recordedFrames.Count)
            {
                StopReplay();
                return null;
            }
```

```
            return null;
        }

        /// <summary>
        /// 保存录制到文件
        /// </summary>
        public void SaveToFile(string filePath)
        {
            try
            {
                using (var writer = new StreamWriter(filePath))
                {
                    writer.WriteLine(recordedFrames.Count);

                    foreach (var frame in recordedFrames)
                    {
                        writer.WriteLine($"{frame.time}|{frame.moveAxis.x}|{fram
e.moveAxis.y}|{frame.lookAxis.x}|{frame.lookAxis.y}");
                        writer.WriteLine(frame.buttonStates.Count);

                        foreach (var kvp in frame.buttonStates)
                        {
                            writer.WriteLine($"{kvp.Key}|{kvp.Value}");
                        }
                    }
                }

                Debug.Log($"[InputRecorder] 保存录制到: {filePath}, 共
{recordedFrames.Count} 帧");
            }
            catch (Exception e)
            {
                Debug.LogError($"[InputRecorder] 保存失败: {e.Message}");
            }
        }

        /// <summary>
        /// 从文件加载录制
        /// </summary>
        public bool LoadFromFile(string filePath)
        {
            try
            {
                if (!File.Exists(filePath))
                {
                    Debug.LogError($"[InputRecorder] 文件不存在: {filePath}");
                    return false;
                }

                recordedFrames.Clear();

                using (var reader = new StreamReader(filePath))
                {
```

```
                        int frameCount = int.Parse(reader.ReadLine());

                        for (int i = 0; i < frameCount; i++)
                        {
                            var frameData = reader.ReadLine().Split('|');
                            float time = float.Parse(frameData[0]);
                            Vector2 moveAxis = new Vector2(float.Parse(frameData[1]),
float.Parse(frameData[2]));
                            Vector2 lookAxis = new Vector2(float.Parse(frameData[3]),
float.Parse(frameData[4]));

                            int buttonCount = int.Parse(reader.ReadLine());
                            var buttonStates = new Dictionary<string, bool>();

                            for (int j = 0; j < buttonCount; j++)
                            {
                                var buttonData = reader.ReadLine().Split('|');
                                buttonStates[buttonData[0]] =
bool.Parse(buttonData[1]);
                            }

                            recordedFrames.Add(new InputFrame
                            {
                                time = time,
                                moveAxis = moveAxis,
                                lookAxis = lookAxis,
                                buttonStates = buttonStates
                            });
                        }
                    }

                    Debug.Log($"[InputRecorder] 加载录制从: {filePath}, 共
{recordedFrames.Count} 帧");
                    return true;
                }
                catch (Exception e)
                {
                    Debug.LogError($"[InputRecorder] 加载失败: {e.Message}");
                    return false;
                }
            }

            /// <summary>
            /// 清除录制
            /// </summary>
            public void Clear()
            {
                recordedFrames.Clear();
                isRecording = false;
                isReplaying = false;
                replayIndex = 0;
            }
```

```
            /// <summary>
            /// 输入帧数据
            /// </summary>
            [System.Serializable]
            public struct InputFrame
            {
                  public float time;
                  public Vector2 moveAxis;
                  public Vector2 lookAxis;
                  public Dictionary<string, bool> buttonStates;
            }
      }
}
using UnityEngine;
using System.Collections.Generic;

namespace XZ.Input
{
      /// <summary>
      /// 键盘+鼠标输入提供者
      /// </summary>
      public class KeyboardInputProvider : IInputProvider
      {
            public InputMode InputMode => InputMode.Keyboard;
            public bool IsActive { get; private set; }
            public float DeadZone { get; set; }

            // 按键映射
            private Dictionary<string, KeyCode> keyMapping = new Dictionary<string, KeyCode>
            {
                  { "Jump", KeyCode.Space },
                  { "Attack", KeyCode.Mouse0 },
                  { "Dash", KeyCode.LeftShift },
                  { "Interact", KeyCode.E },
                  { "Pause", KeyCode.Escape }
            };

            public KeyboardInputProvider(float deadZone = 0.1f)
            {
                  DeadZone = deadZone;
                  IsActive = true;
            }

            public void Update()
            {
                  // 检测是否有键盘输入（排除鼠标）
                  IsActive = IsAnyKeyboardKeyPressed();
            }

            private bool IsAnyKeyboardKeyPressed()
            {
                  // 检查常用键盘按键
                  for (int i = (int)KeyCode.A; i <= (int)KeyCode.Z; i++)
```

```
        {
            if (UnityEngine.Input.GetKey((KeyCode)i))
                return true;
        }

        // 检查数字键
        for (int i = (int)KeyCode.Alpha0; i <= (int)KeyCode.Alpha9; i++)
        {
            if (UnityEngine.Input.GetKey((KeyCode)i))
                return true;
        }

        // 检查方向键
        if (UnityEngine.Input.GetKey(KeyCode.UpArrow) ||
            UnityEngine.Input.GetKey(KeyCode.DownArrow) ||
            UnityEngine.Input.GetKey(KeyCode.LeftArrow) ||
            UnityEngine.Input.GetKey(KeyCode.RightArrow))
            return true;

        // 检查功能键
        if (UnityEngine.Input.GetKey(KeyCode.Space) ||
            UnityEngine.Input.GetKey(KeyCode.LeftShift) ||
            UnityEngine.Input.GetKey(KeyCode.RightShift) ||
            UnityEngine.Input.GetKey(KeyCode.LeftControl) ||
            UnityEngine.Input.GetKey(KeyCode.RightControl) ||
            UnityEngine.Input.GetKey(KeyCode.LeftAlt) ||
            UnityEngine.Input.GetKey(KeyCode.RightAlt) ||
            UnityEngine.Input.GetKey(KeyCode.Escape) ||
            UnityEngine.Input.GetKey(KeyCode.Return) ||
            UnityEngine.Input.GetKey(KeyCode.Tab))
            return true;

        return false;
    }

    public Vector2 GetAxis(string axisName)
    {
        switch (axisName)
        {
            case "Move":
                return GetMoveAxis();

            case "Look":
                return GetLookAxis();

            default:
                return Vector2.zero;
        }
    }

    private Vector2 GetMoveAxis()
    {
        float horizontal = 0f;
```

```
                float vertical = 0f;

                // WASD
                if (UnityEngine.Input.GetKey(KeyCode.W) ||
UnityEngine.Input.GetKey(KeyCode.UpArrow))
                        vertical += 1f;
                if (UnityEngine.Input.GetKey(KeyCode.S) ||
UnityEngine.Input.GetKey(KeyCode.DownArrow))
                        vertical -= 1f;
                if (UnityEngine.Input.GetKey(KeyCode.A) ||
UnityEngine.Input.GetKey(KeyCode.LeftArrow))
                        horizontal -= 1f;
                if (UnityEngine.Input.GetKey(KeyCode.D) ||
UnityEngine.Input.GetKey(KeyCode.RightArrow))
                        horizontal += 1f;

                Vector2 axis = new Vector2(horizontal, vertical);

                // 应用死区
                if (axis.magnitude < DeadZone)
                {
                        return Vector2.zero;
                }

                // 归一化（防止对角线移动过快）
                if (axis.magnitude > 1f)
                {
                        axis.Normalize();
                }

                return axis;
        }

        private Vector2 GetLookAxis()
        {
                // 鼠标移动
                float mouseX = UnityEngine.Input.GetAxis("Mouse X");
                float mouseY = UnityEngine.Input.GetAxis("Mouse Y");

                return new Vector2(mouseX, mouseY);
        }

        public bool GetButton(string buttonName)
        {
                if (keyMapping.ContainsKey(buttonName))
                {
                        return UnityEngine.Input.GetKey(keyMapping[buttonName]);
                }

                return false;
        }

        public bool GetButtonDown(string buttonName)
```

```
            {
                  if (keyMapping.ContainsKey(buttonName))
                  {
                        return UnityEngine.Input.GetKeyDown(keyMapping[buttonName]);
                  }

                  return false;
            }

            public bool GetButtonUp(string buttonName)
            {
                  if (keyMapping.ContainsKey(buttonName))
                  {
                        return UnityEngine.Input.GetKeyUp(keyMapping[buttonName]);
                  }

                  return false;
            }

            /// <summary>
            /// 重新映射按键
            /// </summary>
            public void RemapKey(string buttonName, KeyCode keyCode)
            {
                  keyMapping[buttonName] = keyCode;
            }

            /// <summary>
            /// 获取按键映射
            /// </summary>
            public KeyCode GetKeyMapping(string buttonName)
            {
                  return keyMapping.ContainsKey(buttonName) ? keyMapping[buttonName] :
KeyCode.None;
            }
      }
}
using UnityEngine;

namespace XZ.Skill.Runtime
{
      public interface IEntity
      {
            Object Object { get; }

            bool Destroyed { get; }

            ITransformProxy TransformProxy { get; }

            /// <summary>
            /// 渲染管理
            /// </summary>
            IRenderProxy RenderProxy { get; }
```

```
            /// <summary>
            /// 物理管理
            /// </summary>
            IPhysics Physics { get; }

            /// <summary>
            /// 属性管理
            /// </summary>
            PropertyManager PropertyManager { get; }

            /// <summary>
            /// 动画播放器
            /// </summary>
            public IAnimationPlayer Animation { get; }

            /// <summary>
            /// 类型，用于标记友方或者敌方
            /// </summary>
            int EntityCharacterType { get; }

            // /// <summary>
            // /// 添加 Buff
            // /// </summary>
            // Buff AddBuff(int buffId, IEntity caster);

            // /// <summary>
            // /// 移除 Buff
            // /// </summary>
            // void RemoveBuff(Buff buff);

            /// <summary>
            /// 受到影响
            /// </summary>
            void OnAffect(IEntity other);

            /// <summary>
            /// 更新
            /// </summary>
            void Update(float deltaTime);
        }
}
using System.Collections.Generic;
using FW;

namespace XZ.Skill.Runtime.ECS
{
        /// <summary>
        /// ECS 世界
        /// 管理 EntityManager 和所有 System
        /// </summary>
        public class ECSWorld
        {
```

```
        /// <summary>
        /// Entity 管理器
        /// </summary>
        public EntityManager EntityManager { get; private set; }

        /// <summary>
        /// 系统列表
        /// </summary>
        private List<ISystem> _systems = new List<ISystem>();

        /// <summary>
        /// 世界是否已初始化
        /// </summary>
        public bool IsInitialized { get; private set; }

        public ECSWorld()
        {
            EntityManager = new EntityManager();
            IsInitialized = false;
        }

        /// <summary>
        /// 注册系统
        /// </summary>
        public void RegisterSystem(ISystem system)
        {
            if (system == null)
            {
                Log.Error("[ECS] Cannot register null system");
                return;
            }

            system.Initialize(EntityManager);
            _systems.Add(system);

            Log.Debug($"[ECS] Registered system: {system.GetType().Name}");
        }

        /// <summary>
        /// 移除系统
        /// </summary>
        public void UnregisterSystem(ISystem system)
        {
            if (system == null)
            {
                return;
            }

            if (_systems.Remove(system))
            {
                system.Destroy();
                Log.Debug($"[ECS] Unregistered system: {system.GetType().Name}");
            }
```

```
        }

        /// <summary>
        /// 获取系统
        /// </summary>
        public T GetSystem<T>() where T : class, ISystem
        {
            foreach (var system in _systems)
            {
                if (system is T targetSystem)
                {
                    return targetSystem;
                }
            }
            return null;
        }

        /// <summary>
        /// 初始化世界
        /// </summary>
        public void Initialize()
        {
            if (IsInitialized)
            {
                Log.Warning("[ECS] World already initialized");
                return;
            }

            IsInitialized = true;
            Log.Debug("[ECS] World initialized");
        }

        /// <summary>
        /// 更新世界（更新所有系统）
        /// </summary>
        public void Update(float deltaTime)
        {
            if (!IsInitialized)
            {
                return;
            }

            foreach (var system in _systems)
            {
                system.Update(deltaTime);
            }
        }

        /// <summary>
        /// 销毁世界
        /// </summary>
        public void Destroy()
        {
```

```
                // 销毁所有系统
                for (int i = _systems.Count - 1; i >= 0; i--)
                {
                        _systems[i].Destroy();
                }
                _systems.Clear();

                // 清空 EntityManager
                EntityManager.Clear();

                IsInitialized = false;

                Log.Debug("[ECS] World destroyed");
            }
        }
}
namespace XZ.Skill.Runtime.ECS
{
        /// <summary>
        /// Entity 结构体
        /// Entity 只是一个 ID，不包含任何数据和逻辑
        /// </summary>
        public struct Entity
        {
            /// <summary>
            /// Entity 唯一 ID
            /// </summary>
            public int Id;

            /// <summary>
            /// Entity 版本号
            /// 用于检测 Entity 是否已被销毁
            /// 销毁 Entity 时版本号会递增，使旧引用失效
            /// </summary>
            public int Version;

            /// <summary>
            /// 检查 Entity 是否有效
            /// </summary>
            public bool IsValid(EntityManager manager)
            {
                return manager != null && manager.IsEntityValid(this);
            }

            public override string ToString()
            {
                return $"Entity({Id}.{Version})";
            }

            public override bool Equals(object obj)
            {
                if (obj is Entity other)
                {
```

```
                        return Id == other.Id && Version == other.Version;
                  }
                  return false;
            }

            public override int GetHashCode()
            {
                  return Id.GetHashCode() ^ (Version.GetHashCode() << 16);
            }

            public static bool operator ==(Entity a, Entity b)
            {
                  return a.Id == b.Id && a.Version == b.Version;
            }

            public static bool operator !=(Entity a, Entity b)
            {
                  return !(a == b);
            }
      }
}


using System;
using System.Collections.Generic;
using FW;

namespace XZ.Skill.Runtime.ECS
{
      /// <summary>
      /// Entity 管理器
      /// 负责 Entity 的创建、销毁、组件的添加、移除、查询
      /// </summary>
      public class EntityManager
      {
            private int _nextEntityId = 1;

            // Entity 版本管理：EntityId -> Version
            private Dictionary<int, int> _entityVersions = new Dictionary<int, int>();

            // 组件存储：ComponentType -> EntityId -> Component
            private Dictionary<Type, Dictionary<int, IComponent>> _componentsByType
                  = new Dictionary<Type, Dictionary<int, IComponent>>();

            // 快速查找：EntityId -> HashSet<ComponentType>
            private Dictionary<int, HashSet<Type>> _componentTypesByEntity
                  = new Dictionary<int, HashSet<Type>>();

            /// <summary>
            /// 创建一个新的 Entity
            /// </summary>
            public Entity CreateEntity()
            {
```

```
        int id = _nextEntityId++;
        _entityVersions[id] = 1;
        _componentTypesByEntity[id] = new HashSet<Type>();

        Log.Debug($"[ECS] Created Entity {id}");

        return new Entity { Id = id, Version = 1 };
    }

    /// <summary>
    /// 销毁 Entity
    /// </summary>
    public void DestroyEntity(Entity entity)
    {
        if (!IsEntityValid(entity))
        {
            Log.Warning($"[ECS] Trying to destroy invalid entity {entity}");
            return;
        }

        // 移除所有组件
        if (_componentTypesByEntity.TryGetValue(entity.Id, out var componentTypes))
        {
            foreach (var type in componentTypes)
            {
                if (_componentsByType.TryGetValue(type, out var components))
                {
                    components.Remove(entity.Id);
                }
            }

            _componentTypesByEntity.Remove(entity.Id);
        }

        // 版本号+1，使所有旧引用失效
        _entityVersions[entity.Id]++;

        Log.Debug($"[ECS] Destroyed Entity {entity.Id}");
    }

    /// <summary>
    /// 检查 Entity 是否有效
    /// </summary>
    public bool IsEntityValid(Entity entity)
    {
        return _entityVersions.TryGetValue(entity.Id, out int version)
            && version == entity.Version;
    }

    /// <summary>
    /// 添加组件到 Entity
    /// </summary>
    public void AddComponent<T>(Entity entity, T component) where T : IComponent
```

```
            {
                  if (!IsEntityValid(entity))
                  {
                        Log.Warning($"[ECS] Trying to add component to invalid entity
{entity}");
                        return;
                  }

                  Type type = typeof(T);

                  // 初始化组件存储
                  if (!_componentsByType.ContainsKey(type))
                  {
                        _componentsByType[type] = new Dictionary<int, IComponent>();
                  }

                  // 添加组件
                  _componentsByType[type][entity.Id] = component;
                  _componentTypesByEntity[entity.Id].Add(type);

                  Log.Debug($"[ECS] Added component {type.Name} to Entity {entity.Id}");
            }

            /// <summary>
            /// 从 Entity 移除组件
            /// </summary>
            public void RemoveComponent<T>(Entity entity) where T : IComponent
            {
                  if (!IsEntityValid(entity))
                  {
                        Log.Warning($"[ECS] Trying to remove component from invalid entity
{entity}");
                        return;
                  }

                  Type type = typeof(T);

                  if (_componentsByType.TryGetValue(type, out var components))
                  {
                        components.Remove(entity.Id);
                  }

                  if (_componentTypesByEntity.TryGetValue(entity.Id, out var types))
                  {
                        types.Remove(type);
                  }

                  Log.Debug($"[ECS] Removed component {type.Name} from Entity {entity.Id}");
            }

            /// <summary>
            /// 获取 Entity 的组件
            /// </summary>
```

```
public T GetComponent<T>(Entity entity) where T : IComponent
{
	if (!IsEntityValid(entity))
	{
		return default;
	}

	Type type = typeof(T);

	if (_componentsByType.TryGetValue(type, out var components))
	{
		if (components.TryGetValue(entity.Id, out var component))
		{
			return (T)component;
		}
	}

	return default;
}

/// <summary>
/// 尝试获取 Entity 的组件
/// </summary>
public bool TryGetComponent<T>(Entity entity, out T component) where T : IComponent
{
	component = default;

	if (!IsEntityValid(entity))
	{
		return false;
	}

	Type type = typeof(T);

	if (_componentsByType.TryGetValue(type, out var components))
	{
		if (components.TryGetValue(entity.Id, out var comp))
		{
			component = (T)comp;
			return true;
		}
	}

	return false;
}

/// <summary>
/// 检查 Entity 是否有指定组件
/// </summary>
public bool HasComponent<T>(Entity entity) where T : IComponent
{
	if (!IsEntityValid(entity))
	{
```

```
            return false;
        }

        if (_componentTypesByEntity.TryGetValue(entity.Id, out var types))
        {
            return types.Contains(typeof(T));
        }

        return false;
    }

    /// <summary>
    /// 获取所有拥有指定组件的 Entity 列表
    /// </summary>
    public List<Entity> GetEntitiesWithComponent<T>() where T : IComponent
    {
        var result = new List<Entity>();
        Type type = typeof(T);

        if (_componentsByType.TryGetValue(type, out var components))
        {
            foreach (var entityId in components.Keys)
            {
                if (_entityVersions.TryGetValue(entityId, out int version))
                {
                    result.Add(new Entity { Id = entityId, Version = version });
                }
            }
        }

        return result;
    }

    /// <summary>
    /// 获取所有拥有指定组件的 Entity 列表（多组件筛选）
    /// Entity 必须同时拥有所有指定的组件才会被返回
    /// </summary>
    public List<Entity> GetEntitiesWithComponents(params Type[] componentTypes)
    {
        var result = new List<Entity>();

        if (componentTypes == null || componentTypes.Length == 0)
        {
            return result;
        }

        foreach (var kvp in _componentTypesByEntity)
        {
            int entityId = kvp.Key;
            var entityComponents = kvp.Value;

            // 检查是否包含所有指定组件
            bool hasAll = true;
```

```
                foreach (var type in componentTypes)
                {
                        if (!entityComponents.Contains(type))
                        {
                                hasAll = false;
                                break;
                        }
                }

                if (hasAll && _entityVersions.TryGetValue(entityId, out int version))
                {
                        result.Add(new Entity { Id = entityId, Version = version });
                }
        }

        return result;
}

/// <summary>
/// 获取所有 Entity 的数量
/// </summary>
public int GetEntityCount()
{
        return _componentTypesByEntity.Count;
}

/// <summary>
/// 获取所有有效的 Entity 列表
/// </summary>
public List<Entity> GetAllEntities()
{
        var result = new List<Entity>();

        foreach (var kvp in _entityVersions)
        {
                if (_componentTypesByEntity.ContainsKey(kvp.Key))
                {
                        result.Add(new Entity { Id = kvp.Key, Version = kvp.Value });
                }
        }

        return result;
}

/// <summary>
/// 清空所有 Entity 和组件
/// </summary>
public void Clear()
{
        _entityVersions.Clear();
        _componentsByType.Clear();
        _componentTypesByEntity.Clear();
        _nextEntityId = 1;
```

```
                Log.Debug("[ECS] EntityManager cleared");
            }
        }
}
using System;
using UnityEngine;
using UnityEngine.Audio;

namespace FW
{
    public partial class AudioManager : MonoBehaviour, IAudioManager
    {
        /// <summary>
        /// 音频对应的唯一 Id，每次播放递增
        /// </summary>
        private static int _id = 1;

        private IResManager _resManager;

        private bool _applicationPause;

        private void Update()
        {
            UpdateAgent();
            UpdateBGM();
            UpdateRhythmChannels();
        }

        private void OnApplicationPause(bool pauseStatus)
        {
            _applicationPause = pauseStatus;
        }

        #region interface

        void IAudioManager.Init(IResManager resManager, IGameObjectPoolManager
gameObjectPoolManager)
        {
            _resManager = resManager;

            InitPool(gameObjectPoolManager);

            InitGroups();
        }

        int IAudioManager.Play(string audioName, string audioGroupName)
        {
            var id = _id++;
            var audioMixerGroup = GetGroup(audioGroupName);

            _resManager.LoadAssetAsync<AudioClip>(audioName, (audioClip) =>
            {
```

```
                OnLoadSuccess(id, audioClip, audioMixerGroup);
            });

            return id;
        }

        int IAudioManager.PlayReplace(string audioName, string audioGroupName, string
replaceKey)
        {
            if (!string.IsNullOrEmpty(replaceKey))
                StopAndRemoveAgentsByReplaceKey(replaceKey);
            var id = _id++;
            var audioMixerGroup = GetGroup(audioGroupName);
            _resManager.LoadAssetAsync<AudioClip>(audioName, (audioClip) =>
            {
                OnLoadSuccessReplace(id, audioClip, audioMixerGroup, replaceKey);
            });
            return id;
        }

        int IAudioManager.PlayWithPitch(string audioName, string audioGroup, float volume,
float pitch)
        {
            var id = _id++;
            var audioMixerGroup = GetGroup(audioGroup);

            _resManager.LoadAssetAsync<AudioClip>(audioName, (audioClip) =>
            {
                OnLoadSuccess(id, audioClip, audioMixerGroup, volume, pitch);
            });

            return id;
        }

        int IAudioManager.PlayScheduledWithPitch(string audioName, string audioGroup,
float volume, float pitch, double dspTime)
        {
            var id = _id++;
            var audioMixerGroup = GetGroup(audioGroup);

            _resManager.LoadAssetAsync<AudioClip>(audioName, (audioClip) =>
            {
                OnLoadSuccessScheduled(id, audioClip, audioMixerGroup, volume, pitch,
dspTime);
            });

            return id;
        }

        int IAudioManager.PlayMelodicShot(string audioName, string audioGroup)
        {
            var pitch = GetRandomMelodicPitch();
```

```
            return ((IAudioManager)this).PlayWithPitch(audioName, audioGroup, 1f,
pitch);
        }

        int IAudioManager.PlayScheduledMelodicShot(string audioName, string audioGroup,
double dspTime)
        {
            var pitch = GetRandomMelodicPitch();
            return ((IAudioManager)this).PlayScheduledWithPitch(audioName, audioGroup,
1f, pitch, dspTime);
        }

        void IAudioManager.Pause(int id)
        {
            GetAgent(id)?.Pause();
        }

        void IAudioManager.Resume(int id)
        {
            GetAgent(id)?.Resume();
        }

        void IAudioManager.Stop(int id)
        {
            GetAgent(id).Stop();
        }

        void IAudioManager.SetVolume(string audioGroup, float volume)
        {
            var groupInfo = GetGroupInfo(audioGroup);
            if (groupInfo == null)
            {
                return;
            }

            var groupVolume = FuncGroupVolume(volume);
            audioMixer.SetFloat(groupInfo.VolumeParamName, groupVolume);
        }

        float IAudioManager.GetVolume(string audioGroup)
        {
            float volume = 0;
            var groupInfo = GetGroupInfo(audioGroup);
            if (groupInfo == null)
            {
                return volume;
            }

            audioMixer.GetFloat(groupInfo.VolumeParamName, out volume);
            return RevertFuncGroupVolume(volume);
        }

        void IAudioManager.SetRhythmTiming(float bpm, int beatsPerBar)
```

```
        {
            SetRhythmTimingInternal(bpm, beatsPerBar);
        }

        void IAudioManager.StartRhythm(double delaySeconds)
        {
            StartRhythmInternal(delaySeconds);
        }

        double IAudioManager.GetNextBarEntryTime()
        {
            return GetNextBarEntryTimeInternal();
        }

        double IAudioManager.GetNextQuantizedTime(double intervalSeconds)
        {
            return GetNextQuantizedTimeInternal(intervalSeconds);
        }

        int IAudioManager.CreateRhythmChannel(string audioGroupName, float volume)
        {
            return CreateRhythmChannelInternal(audioGroupName, volume);
        }

        void IAudioManager.ChangeRhythmTrack(int channelId, string audioResName, bool loop,
bool crossfade, float crossfadeSeconds, bool quantize)
        {
            ChangeRhythmTrackInternal(channelId, audioResName, loop, crossfade,
crossfadeSeconds, quantize);
        }

        void IAudioManager.StopRhythmChannel(int channelId)
        {
            StopRhythmChannelInternal(channelId);
        }

        private void OnLoadSuccess(int id, AudioClip audioClip, AudioMixerGroup mixerGroup)
        {
            var go = _gameObjectPool.Get();
            var audioSource = go.GetComponent<AudioSource>();
            audioSource.playOnAwake = false;
            audioSource.loop = false;
            audioSource.clip = audioClip;
            audioSource.volume = 1f;
            audioSource.pitch = 1f;
            audioSource.outputAudioMixerGroup = mixerGroup;

            var agent = Agent.Create(id, audioSource);
            _agents.Add(agent);

            agent.Play();
        }
```

```
            private void OnLoadSuccessReplace(int id, AudioClip audioClip, AudioMixerGroup
mixerGroup, string replaceKey)
            {
                  var go = _gameObjectPool.Get();
                  var audioSource = go.GetComponent<AudioSource>();
                  audioSource.playOnAwake = false;
                  audioSource.loop = false;
                  audioSource.clip = audioClip;
                  audioSource.volume = 1f;
                  audioSource.pitch = 1f;
                  audioSource.outputAudioMixerGroup = mixerGroup;

                  var agent = Agent.Create(id, audioSource);
                  agent.ReplaceKey = replaceKey;
                  _agents.Add(agent);

                  agent.Play();
            }

            private void OnLoadSuccess(int id, AudioClip audioClip, AudioMixerGroup mixerGroup,
float volume, float pitch)
            {
                  var go = _gameObjectPool.Get();
                  var audioSource = go.GetComponent<AudioSource>();
                  audioSource.playOnAwake = false;
                  audioSource.loop = false;
                  audioSource.clip = audioClip;
                  audioSource.volume = Mathf.Clamp01(volume);
                  audioSource.pitch = pitch;
                  audioSource.outputAudioMixerGroup = mixerGroup;

                  var agent = Agent.Create(id, audioSource);
                  _agents.Add(agent);

                  agent.Play();
            }

            private void OnLoadSuccessScheduled(int id, AudioClip audioClip, AudioMixerGroup
mixerGroup, float volume, float pitch, double dspTime)
            {
                  var go = _gameObjectPool.Get();
                  var audioSource = go.GetComponent<AudioSource>();
                  audioSource.playOnAwake = false;
                  audioSource.loop = false;
                  audioSource.clip = audioClip;
                  audioSource.volume = Mathf.Clamp01(volume);
                  audioSource.pitch = pitch;
                  audioSource.outputAudioMixerGroup = mixerGroup;

                  var agent = Agent.Create(id, audioSource);
                  _agents.Add(agent);

                  double now = AudioSettings.dspTime;
```

```
                if (dspTime <= now + 0.001f)
                {
                      agent.Play();
                }
                else
                {
                      agent.PlayScheduled(dspTime);
                }
          }

          #endregion
     }
}
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Audio;

namespace FW
{
     public partial class AudioManager
     {
          [SerializeField] private AudioMixer audioMixer;

          [Serializable]
          private class GroupInfo
          {
                /// <summary>
                /// 组名
                /// </summary>
                public string Name;

                /// <summary>
                /// 设置音量参数（需要在编辑器中暴露出来）
                /// </summary>
                public string VolumeParamName;
          }

          /// <summary>
          /// 最低音量
          /// </summary>
          [SerializeField] private float minAttenuation = -80;

          /// <summary>
          /// 最高音量
          /// 最高可以到20，但是超过0之后会有爆音，所有不用
          /// </summary>
          [SerializeField] private float maxAttenuation = 0;

          /// <summary>
          /// 组信息（mixer拿不到，只能自己组织，比较屎）
          /// </summary>
          [SerializeField] private GroupInfo[] groupInfos;
```

```
private readonly Dictionary<string, AudioMixerGroup> _mixerGroups = new();
private float _attenuationThreshold;

private void InitGroups()
{
	_attenuationThreshold = maxAttenuation - minAttenuation;

	foreach (var groupInfo in groupInfos)
	{
		var groupName = groupInfo.Name;
		var groups = audioMixer.FindMatchingGroups(groupName);

		foreach (var group in groups)
		{
			if (group.name == groupName)
			{
				_mixerGroups.Add(groupName, group);
				break;
			}
		}
	}
}

private AudioMixerGroup GetGroup(string groupName)
{
	_mixerGroups.TryGetValue(groupName, out var group);
	return group;
}

private GroupInfo GetGroupInfo(string groupName)
{
	foreach (var groupInfo in groupInfos)
	{
		if (groupInfo.Name == groupName)
		{
			return groupInfo;
		}
	}

	return null;
}

/// <summary>
/// 映射到自定义空间，音量太高的话会有爆音
/// </summary>
/// <param name="percent"></param>
/// <returns></returns>
private float FuncGroupVolume(float percent)
{
	return minAttenuation + _attenuationThreshold * percent;
}
```

```
            /// <summary>
            /// 音量映射回 0-1
            /// </summary>
            /// <param name="volume"></param>
            /// <returns></returns>
            private float RevertFuncGroupVolume(float volume)
            {
                  return (volume - minAttenuation) / _attenuationThreshold;
            }
      }
}
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Audio;

namespace FW
{
      public partial class AudioManager
      {
            private class RhythmChannel : IReference
            {
                  public int Id;
                  public AudioSource SourceA;
                  public AudioSource SourceB;
                  public AudioClip ClipA;
                  public AudioClip ClipB;
                  public bool ClipAFromRes;
                  public bool ClipBFromRes;
                  public double EndTimeA;
                  public double EndTimeB;
                  public bool UsingA = true;
                  public float Volume = 1f;
                  public AudioMixerGroup OutputGroup;
                  public int LoadVersion;

                  public void Clear()
                  {
                        Id = 0;
                        SourceA = null;
                        SourceB = null;
                        ClipA = null;
                        ClipB = null;
                        ClipAFromRes = false;
                        ClipBFromRes = false;
                        EndTimeA = 0d;
                        EndTimeB = 0d;
                        UsingA = true;
                        Volume = 1f;
                        OutputGroup = null;
                        LoadVersion = 0;
                  }
            }
```

```
        private struct FadeJob
        {
            public AudioSource Source;
            public float From;
            public float To;
            public float Duration;
            public float StartTime;
        }

        private static readonly float[] s_MelodicPitchTable =
        {
            1f, 1.189f, 1.335f, 1.498f, 1.782f, 2f
        };

        // private static readonly float[] s_MelodicPitchTable =
        // {
        //     1.0f
        // };

        private readonly List<RhythmChannel> _rhythmChannels = new(8);
        private readonly List<FadeJob> _fadeJobs = new(8);
        private int _rhythmChannelId = 1;

        private float _rhythmBpm = 120f;
        private int _rhythmBeatsPerBar = 4;
        private double _rhythmStartDspTime;
        private bool _rhythmStarted;

        private void SetRhythmTimingInternal(float bpm, int beatsPerBar)
        {
            _rhythmBpm = Mathf.Max(1f, bpm);
            _rhythmBeatsPerBar = Mathf.Max(1, beatsPerBar);
        }

        private void StartRhythmInternal(double delaySeconds)
        {
            _rhythmStartDspTime = AudioSettings.dspTime + Mathf.Max(0f,
(float)delaySeconds);
            _rhythmStarted = true;
        }

        private double GetNextBarEntryTimeInternal()
        {
            if (!_rhythmStarted)
            {
                StartRhythmInternal(0);
            }

            return GetNextQuantizedTimeInternal(SecPerBar);
        }

        private double GetNextQuantizedTimeInternal(double intervalSeconds)
        {
```

```
            if (intervalSeconds <= 0.0)
            {
                return AudioSettings.dspTime;
            }

            if (!_rhythmStarted)
            {
                StartRhythmInternal(0);
            }

            double dsp = AudioSettings.dspTime;
            double elapsed = dsp - _rhythmStartDspTime;
            if (elapsed <= 0.0)
            {
                return _rhythmStartDspTime;
            }

            long steps = (long)Mathf.Floor((float)(elapsed / intervalSeconds)) + 1;
            return _rhythmStartDspTime + steps * intervalSeconds;
        }

        private double SecPerBeat => 60.0 / _rhythmBpm;
        private double SecPerBar => SecPerBeat * _rhythmBeatsPerBar;

        private int CreateRhythmChannelInternal(string audioGroupName, float volume)
        {
            var sourceB = CreateRhythmSource();
            var sourceA = CreateRhythmSource();
            var group = GetGroup(audioGroupName);

            sourceA.outputAudioMixerGroup = group;
            sourceB.outputAudioMixerGroup = group;

            var channel = ReferencePool.Acquire<RhythmChannel>();
            channel.Id = _rhythmChannelId++;
            channel.SourceA = sourceA;
            channel.SourceB = sourceB;
            channel.Volume = Mathf.Clamp01(volume);
            channel.OutputGroup = group;

            _rhythmChannels.Add(channel);
            return channel.Id;
        }

        private void ChangeRhythmTrackInternal(int channelId, string audioResName, bool
loop, bool crossfade, float crossfadeSeconds, bool quantize)
        {
            var channel = GetRhythmChannel(channelId);
            if (channel == null)
            {
                return;
            }
```

```
            channel.LoadVersion++;
            int version = channel.LoadVersion;

            _resManager.LoadAssetAsync<AudioClip>(audioResName, (clip) =>
            {
                if (clip == null)
                {
                    return;
                }

                var currentChannel = GetRhythmChannel(channelId);
                if (currentChannel == null || currentChannel.LoadVersion != version)
                {
                    _resManager.Release(clip);
                    return;
                }

                SwitchRhythmClip(currentChannel, clip, loop, crossfade,
crossfadeSeconds, true, quantize);
            });
        }

        private void StopRhythmChannelInternal(int channelId)
        {
            for (int i = _rhythmChannels.Count - 1; i >= 0; i--)
            {
                var channel = _rhythmChannels[i];
                if (channel.Id != channelId)
                {
                    continue;
                }

                StopRhythmChannel(channel);
                _rhythmChannels.RemoveAt(i);
                return;
            }
        }

        private void StopRhythmChannel(RhythmChannel channel)
        {
            StopRhythmSource(channel.SourceA);
            StopRhythmSource(channel.SourceB);
            ReleaseClip(ref channel.ClipA, ref channel.ClipAFromRes);
            ReleaseClip(ref channel.ClipB, ref channel.ClipBFromRes);

            ReferencePool.Release(channel);
        }

        private AudioSource CreateRhythmSource()
        {
            var go = _gameObjectPool.Get();
            var source = go.GetComponent<AudioSource>();
            source.playOnAwake = false;
```

```
            source.loop = true;
            source.volume = 1f;
            source.pitch = 1f;
            return source;
        }

        private void SwitchRhythmClip(RhythmChannel channel, AudioClip newClip, bool loop,
bool crossfade, float crossfadeSeconds, bool fromRes, bool quantize)
        {
            double switchTime = quantize
                ? GetNextBarEntryTimeInternal()
                : AudioSettings.dspTime + 0.02f;
            var current = channel.UsingA ? channel.SourceA : channel.SourceB;
            var next = channel.UsingA ? channel.SourceB : channel.SourceA;

            if (current.isPlaying)
            {
                current.SetScheduledEndTime(switchTime);
                SetEndTime(channel, current, switchTime);
            }

            PrepareRhythmSource(channel, next, newClip, fromRes);
            next.loop = loop;
            next.volume = channel.Volume;

            if (crossfade && crossfadeSeconds > 0f)
            {
                next.volume = 0f;
                next.PlayScheduled(switchTime);
                StartFade(next, 0f, channel.Volume, crossfadeSeconds);
                if (current.isPlaying)
                {
                    StartFade(current, current.volume, 0f, crossfadeSeconds);
                }
            }
            else
            {
                next.PlayScheduled(switchTime);
            }

            if (!loop)
            {
                SetEndTime(channel, next, switchTime + newClip.length);
            }

            channel.UsingA = !channel.UsingA;
        }

        private void PrepareRhythmSource(RhythmChannel channel, AudioSource source,
AudioClip clip, bool fromRes)
        {
            if (source == channel.SourceA)
            {
```

```
                        ReplaceClip(ref channel.ClipA, ref channel.ClipAFromRes, clip,
fromRes);
                        channel.EndTimeA = 0d;
                  }
                  else
                  {
                        ReplaceClip(ref channel.ClipB, ref channel.ClipBFromRes, clip,
fromRes);
                        channel.EndTimeB = 0d;
                  }

                  source.clip = clip;
                  source.outputAudioMixerGroup = channel.OutputGroup;
            }

            private void ReplaceClip(ref AudioClip oldClip, ref bool oldFromRes, AudioClip
newClip, bool newFromRes)
            {
                  if (oldClip != null && oldFromRes)
                  {
                        _resManager.Release(oldClip);
                  }

                  oldClip = newClip;
                  oldFromRes = newFromRes;
            }

            private void ReleaseClip(ref AudioClip clip, ref bool fromRes)
            {
                  if (clip != null && fromRes)
                  {
                        _resManager.Release(clip);
                  }

                  clip = null;
                  fromRes = false;
            }

            private void StopRhythmSource(AudioSource source)
            {
                  if (source == null)
                  {
                        return;
                  }

                  source.Stop();
                  source.clip = null;
                  _gameObjectPool.Release(source.gameObject);
            }

            private void SetEndTime(RhythmChannel channel, AudioSource source, double endTime)
            {
                  if (source == channel.SourceA)
```

```
                {
                        channel.EndTimeA = endTime;
                }
                else
                {
                        channel.EndTimeB = endTime;
                }
        }

        private void UpdateRhythmChannels()
        {
                UpdateFadeJobs();

                if (_rhythmChannels.Count == 0)
                {
                        return;
                }

                for (int i = _rhythmChannels.Count - 1; i >= 0; i--)
                {
                        var channel = _rhythmChannels[i];
                        CheckRelease(channel.SourceA, ref channel.ClipA, ref
channel.ClipAFromRes, channel.EndTimeA);
                        CheckRelease(channel.SourceB, ref channel.ClipB, ref
channel.ClipBFromRes, channel.EndTimeB);
                }
        }

        private void CheckRelease(AudioSource source, ref AudioClip clip, ref bool fromRes,
double endTime)
        {
                if (clip == null || !fromRes)
                {
                        return;
                }

                if (!source.isPlaying && endTime > 0.0 && AudioSettings.dspTime > endTime +
0.05f)
                {
                        _resManager.Release(clip);
                        clip = null;
                        fromRes = false;
                }
        }

        private void StartFade(AudioSource source, float from, float to, float duration)
        {
                if (source == null)
                {
                        return;
                }

                _fadeJobs.Add(new FadeJob
```

```
            {
                Source = source,
                From = from,
                To = to,
                Duration = Mathf.Max(0.01f, duration),
                StartTime = Time.unscaledTime
            });
        }

        private void UpdateFadeJobs()
        {
            if (_fadeJobs.Count == 0)
            {
                return;
            }

            for (int i = _fadeJobs.Count - 1; i >= 0; i--)
            {
                var job = _fadeJobs[i];
                if (job.Source == null)
                {
                    _fadeJobs.RemoveAt(i);
                    continue;
                }

                float t = (Time.unscaledTime - job.StartTime) / job.Duration;
                if (t >= 1f)
                {
                    job.Source.volume = job.To;
                    _fadeJobs.RemoveAt(i);
                }
                else
                {
                    job.Source.volume = Mathf.Lerp(job.From, job.To, t);
                }
            }
        }

        private RhythmChannel GetRhythmChannel(int channelId)
        {
            for (int i = 0; i < _rhythmChannels.Count; i++)
            {
                if (_rhythmChannels[i].Id == channelId)
                {
                    return _rhythmChannels[i];
                }
            }

            return null;
        }

        private float GetRandomMelodicPitch()
        {
```

```
                float bgmPitch = GetCurrentBgmPitch();
                return bgmPitch;
            }

            private float GetCurrentBgmPitch()
            {
                for (int i = 0; i < _rhythmChannels.Count; i++)
                {
                    var channel = _rhythmChannels[i];
                    if (channel == null)
                    {
                        continue;
                    }

                    var sourceA = channel.SourceA;
                    if (sourceA != null && sourceA.isPlaying)
                    {
                        return sourceA.pitch;
                    }

                    var sourceB = channel.SourceB;
                    if (sourceB != null && sourceB.isPlaying)
                    {
                        return sourceB.pitch;
                    }

                    var preferred = channel.UsingA ? sourceA : sourceB;
                    if (preferred != null)
                    {
                        return preferred.pitch;
                    }
                }

                return 1f;
            }

        }
}
using System;
using Cysharp.Threading.Tasks;
using Events;
using FW;
using Model.UI;
using Presenter.Utils;
using UnityEngine;
using TMPro;

namespace Presenter.UI
{
    public partial class CopyText
    {
        private const float ScreenOffsetY = 400f;
        private const float FloatSpeedPixelsPerSecond = 280f;
```

```
            private const float Duration = 1.2f;
            private const string CopyTextPoolName = "UICopyText_Item";

            private Transform _templateTransform;
            private IGameObjectPool _copyTextPool;
            private EventHandler<BaseEventArgs> _copyTextEventHandler;

            protected override void Open()
            {
                  if (_templateTransform == null)
                  {
                        _templateTransform = _view.CopyTextUI;
                        _templateTransform.gameObject.SetActive(false);

                        _copyTextEventHandler = OnCopyTextEvent;
                        Game.Event.Subscribe(CopyTextEventArgs.EventId,
_copyTextEventHandler);
                  }

                  if (_copyTextPool == null)
                        _copyTextPool = Game.GOPool.Create(CopyTextPoolName,
_templateTransform.gameObject);

                  if (_model != null && !string.IsNullOrEmpty(_model.CopyText))
                        SpawnAndAnimateCopy(_model.CopyText, _model.ScreenPosition);
            }

            protected override void Close()
            {
                  if (_copyTextEventHandler != null)
                  {
                        Game.Event.Unsubscribe(CopyTextEventArgs.EventId,
_copyTextEventHandler);
                        _copyTextEventHandler = null;
                  }
                  if (_copyTextPool != null)
                  {
                        Game.GOPool.Destroy(CopyTextPoolName);
                        _copyTextPool = null;
                  }
                  base.Close();
            }

            private void OnCopyTextEvent(object sender, BaseEventArgs e)
            {
                  if (e is not CopyTextEventArgs args || string.IsNullOrEmpty(args.CopyText))
return;
                  SpawnAndAnimateCopy(args.CopyText, args.ScreenPosition);
            }

            private void SpawnAndAnimateCopy(string text, Vector2 screenPos)
            {
                  var clone = _copyTextPool.Get();
```

```
            clone.transform.SetParent(ViewBase.Go.transform, false);

            var cloneRect = clone.GetComponent<RectTransform>();
            var tmp = clone.GetComponentInChildren<TextMeshProUGUI>();
            if (cloneRect == null || tmp == null)
            {
                _copyTextPool.Release(clone);
                return;
            }

            tmp.text = text;
            tmp.color = new Color(1f, 1f, 1f, 1f);

            screenPos.y += ScreenOffsetY;
            PositionRectAtScreen(cloneRect, screenPos);

            AnimateCloneAndReleaseAsync(clone, cloneRect, tmp, screenPos).Forget();
        }

        private void PositionRectAtScreen(RectTransform rect, Vector2 screenPos)
        {
            var parentRect = ViewBase.Go.GetComponent<RectTransform>();
            if (parentRect == null) return;

            var canvas = ViewBase.Go.GetComponent<Canvas>();
            var cam = canvas != null ? canvas.worldCamera : null;
            if (cam == null && canvas != null)
                cam = canvas.rootCanvas != null ? canvas.rootCanvas.worldCamera : null;

            RectTransformUtility.ScreenPointToLocalPointInRectangle(parentRect,
screenPos, cam, out Vector2 local);
            // 父节点 pivot 在左下，local 以左下为原点；子节点 anchor 在 (0.5,0.5) 时
anchoredPosition 是相对父节点中心的偏移
            Vector2 parentCenter = new Vector2(parentRect.rect.width * 0.5f,
parentRect.rect.height * 0.5f);
            rect.anchoredPosition = local - parentCenter;
        }

        private async UniTaskVoid AnimateCloneAndReleaseAsync(GameObject clone,
RectTransform cloneRect, TextMeshProUGUI tmp, Vector2 screenPos)
        {
            float elapsed = 0f;
            while (elapsed < Duration)
            {
                elapsed += Time.deltaTime;
                float t = elapsed / Duration;
                screenPos.y += FloatSpeedPixelsPerSecond * Time.deltaTime;
                PositionRectAtScreen(cloneRect, screenPos);
                float a = 1f - Mathf.Clamp01(t * 1.5f);
                tmp.color = new Color(1f, 1f, 1f, a);
                await UniTask.Yield();
            }
```

```
                _copyTextPool.Release(clone);
            }

            /// <summary>
            /// 按角色取随机文案并通过事件在指定屏幕坐标显示（传 screenPos，界面与场景相机不同）
            /// </summary>
            public static void ShowCopyFor(CopyTextConfig.RoleId role, Vector2 screenPosition)
            {
                string copy = CopyTextConfig.GetRandomCopy(role);
                if (string.IsNullOrEmpty(copy)) return;
                Game.Event.Fire(null, CopyTextEventArgs.Create(copy, screenPosition));
            }
        }
}
using System.Collections;
using UnityEngine;

public class MuYuChuiCollider : MonoBehaviour
{
        [SerializeField] private string _knockSoundName = "SFX_MuYu_Knock_02";
        [SerializeField] private string _knockSoundGroupName = "Rhythm_High";
        [SerializeField] private float _knockSoundCooldown = 0.1f;
        [Header("Debug: 碰撞点 worldPos 可视化")]
        [SerializeField] private bool _debugCollisionPoint;
        [SerializeField] private float _debugMarkerDuration = 2f;
        [SerializeField] private float _debugMarkerSize = 0.8f;
        private float _lastKnockTime = float.NegativeInfinity;

        private void OnCollisionEnter2D(Collision2D collision)
        {
            if (Time.time - _lastKnockTime < _knockSoundCooldown)
                return;
            _lastKnockTime = Time.time;

            Game.Audio.PlayReplace(_knockSoundName, _knockSoundGroupName, "MuYuKnock");

            Vector3 worldPos = collision.contactCount > 0
                ? (Vector3)collision.GetContact(0).point
                : transform.position;

            if (_debugCollisionPoint)
                StartCoroutine(ShowDebugMarkerAt(worldPos));

            Vector2 screenPos = Camera.main != null
                ? (Vector2)Camera.main.WorldToScreenPoint(worldPos)
                : Vector2.zero;
            Presenter.UI.CopyText.ShowCopyFor(CopyTextConfig.RoleId.MuYu, screenPos);
        }

        private IEnumerator ShowDebugMarkerAt(Vector3 worldPos)
        {
            var go = new GameObject("DebugCollisionPoint");
```

```
            go.transform.position = worldPos;

            var tex = new Texture2D(16, 16);
            for (int y = 0; y < 16; y++)
                  for (int x = 0; x < 16; x++)
                        tex.SetPixel(x, y, (x - 8) * (x - 8) + (y - 8) * (y - 8) <= 64 ? Color.red :
Color.clear);
            tex.Apply();

            var sr = go.AddComponent<SpriteRenderer>();
            sr.sprite = Sprite.Create(tex, new Rect(0, 0, 16, 16), new Vector2(0.5f, 0.5f));
            sr.sortingOrder = 32767;
            go.transform.localScale = Vector3.one * _debugMarkerSize;

            yield return new WaitForSeconds(_debugMarkerDuration);
            if (sr != null && sr.sprite != null) Destroy(sr.sprite);
            Destroy(tex);
            Destroy(go);
      }
}

using FW;
using UnityEngine;

namespace Events
{
      public class CopyTextEventArgs : BaseEventArgs
      {
            public static readonly int EventId = typeof(CopyTextEventArgs).GetHashCode();

            public override int Id => EventId;

            public string CopyText { get; private set; }
            /// <summary>屏幕坐标，用于界面定位（界面与场景用不同相机，应传 screenPos）
</summary>
            public Vector2 ScreenPosition { get; private set; }

            public static CopyTextEventArgs Create(string copyText, Vector2 screenPosition)
            {
                  var args = ReferencePool.Acquire<CopyTextEventArgs>();
                  args.CopyText = copyText;
                  args.ScreenPosition = screenPosition;
                  return args;
            }

            public override void Clear()
            {
                  CopyText = null;
                  ScreenPosition = default;
            }
      }
}
```

```
using FW;

namespace Events
{
	public class GameSelectEventArgs : BaseEventArgs
	{
		public static readonly int EventId = typeof(GameSelectEventArgs).GetHashCode();

		public override int Id => EventId;

		public int GameId { get; private set; }

		public static GameSelectEventArgs Create(int characterId)
		{
			var args = ReferencePool.Acquire<GameSelectEventArgs>();
			args.GameId = characterId;
			return args;
		}

		public override void Clear()
		{
			GameId = 0;
		}
	}
}
using Cysharp.Threading.Tasks;
using FW;
using FW.Procedure;
using Model.UI;
using Presenter.UI;
using Presenter.Utils;

namespace Presenter
{
	public class ProcedureStart : ProcedureBase
	{
		private IFSM<IProcedureManager> _fsm;

		protected override void OnInit(IFSM<IProcedureManager> fsm)
		{
			Log.Debug("Procedure start init");

			_fsm = fsm;
		}

		protected override void OnEnter(IFSM<IProcedureManager> fsm)
		{
			Log.Debug("Procedure start enter");

			Game.Event.Subscribe(Events.GameSelectEventArgs.EventId, OnGameSelect);

			UniTask.Create(DoStart);
		}
```

```
            protected override void OnLeave(IFSM<IProcedureManager> fsm, bool isDestroy)
            {
                  Game.Event.Unsubscribe(Events.GameSelectEventArgs.EventId, OnGameSelect);
            }

            private async UniTask DoStart()
            {
                  Game.Audio.PlayBGM("BGM_room-tone-bano-ventiladores");

                  await Game.UI.Open<Start>(UINames.Start, StartModel.Create());
            }

            private void OnGameSelect(object sender, BaseEventArgs args)
            {
                  var gameSelectEventArgs = args as Events.GameSelectEventArgs;
                  if (gameSelectEventArgs == null)
                  {
                        Log.Error("Procedure start game select: args is null");
                        return;
                  }

                  Log.Debug($"Procedure start game select: {gameSelectEventArgs.GameId}");

                  Game.Procedure.SetProcedureData(gameSelectEventArgs.GameId);
                  _fsm.ChangeState<ProcedureGame>();

                  Game.UI.Close(UINames.Start);
            }
      }
}
using FW;
using FW.Procedure;
using Cysharp.Threading.Tasks;
using Presenter.Utils;
using Model.UI;
using Presenter.UI;

namespace Presenter
{
      public class ProcedureGame : ProcedureBase
      {
            private IFSM<IProcedureManager> _fsm;

            private int _gameId;

            protected override void OnInit(IFSM<IProcedureManager> fsm)
            {
                  Log.Debug("Procedure game init");

                  _fsm = fsm;
            }
```

```
            protected override void OnEnter(IFSM<IProcedureManager> fsm, object data)
            {
                  _gameId = data is int id ? id : 0;
                  Log.Debug($"Procedure game enter, GameId: {_gameId}");

                  UniTask.Create(DoEnter);
            }

            protected override void OnLeave(IFSM<IProcedureManager> fsm, bool isDestroy)
            {
                  Log.Debug("Procedure game leave");
            }

            private async UniTask DoEnter()
            {

                  switch (_gameId)
                  {
                        case 1:
                              await Game.Res.LoadSceneAsync("GameXianYu");
                              Game.Audio.PlayBGM("BGM_XianYu");
                              break;
                        case 2:
                              await Game.Res.LoadSceneAsync("GameMuYu");
                              Game.Audio.PlayBGM("BGM_MuYu");
                              break;
                        default:
                              Log.Error("Procedure start game select: game id is not found");
                              break;
                  }

                  await Game.UI.Open<CopyText>(UINames.CopyText, CopyTextModel.Create());
            }
      }
}
using System;
using System.Collections.Generic;

namespace FW
{
      public static partial class ReferencePool
      {
            private sealed class ReferenceCollection
            {
                  private readonly Queue<IReference> _References;
                  private readonly Type _ReferenceType;

                  public int UsingCount { get; private set; }
                  public int AcquireCount { get; private set; }
                  public int ReleaseCount { get; private set; }
                  public int AddCount { get; private set; }
                  public int RemoveCount { get; private set; }
```

```
public ReferenceCollection(Type referenceReferenceType)
{
	_References = new Queue<IReference>();
	_ReferenceType = referenceReferenceType;
	UsingCount = 0;
	AcquireCount = 0;
	ReleaseCount = 0;
	AddCount = 0;
	RemoveCount = 0;
}

public Type ReferenceType => _ReferenceType;

public int UnusedCount
{
	get
	{
		return _References.Count;
	}
}

public T Acquire<T>() where T : class, IReference, new()
{
	if (typeof(T) != _ReferenceType)
	{
		throw new Exception("Type is invalid.");
	}

	UsingCount++;
	AcquireCount++;

	lock (_References)
	{
		if (_References.Count > 0)
		{
			return (T)_References.Dequeue();
		}
	}

	AddCount++;

	return new T();
}

public void Release(IReference reference)
{
	reference.Clear();

	lock (_References)
	{
		if (_References.Contains(reference))
		{
			throw new Exception("The reference has been released.");
```

```
                        }

                        _References.Enqueue(reference);
                    }

                    ReleaseCount++;
                    UsingCount--;
                }

                public void ReleaseAll()
                {
                    lock (_References)
                    {
                        RemoveCount += _References.Count;
                        _References.Clear();
                    }
                }
            }
        }
}
using System;
using System.Collections.Generic;

namespace FW
{
    public static partial class ReferencePool
    {
        private static readonly Dictionary<Type, ReferenceCollection> _Collections =
            new Dictionary<Type, ReferenceCollection>();

        public static int Count => _Collections.Count;

        public static T Acquire<T>() where T : class, IReference, new ()
        {
            return GetCollection(typeof(T)).Acquire<T>();
        }

        public static void Release(IReference reference)
        {
            if (reference == null)
            {
                throw new Exception("Reference pool release reference is invalid.");
            }

            var type = reference.GetType();
            GetCollection(type).Release(reference);
        }

        public static void Clear()
        {
            lock (_Collections)
            {
                foreach (var collection in _Collections)
```

```
                {
                    collection.Value.ReleaseAll();
                }

                _Collections.Clear();
            }
        }

        private static ReferenceCollection GetCollection(Type referenceType)
        {
            if (referenceType == null)
            {
                throw new Exception("Reference type is invalid.");
            }

            ReferenceCollection collection;
            lock (_Collections)
            {
                if (!_Collections.TryGetValue(referenceType, out collection))
                {
                    collection = new ReferenceCollection(referenceType);
                    _Collections.Add(referenceType, collection);
                }
            }

            return collection;
        }
    }
}
```